

INTERNSHIP REPORT

On

Design and Implementation of Forward Collision Warning System using Model-Based Design Workflow in MATLAB/Simulink

Submitted in partial fulfilment of the requirements for the Internship
Programme at

KPIT Apex Lab

Duration: 8 Weeks

Mentor: Shubhada Ma'am

Submitted By

Swapnil Kumbhar

Omkar Meher

Prathamesh Lokhande

Internship Duration: _____

Department of Engineering
Academic Year 2025–2026

ABSTRACT

Forward collision accidents account for a significant proportion of road accidents, making Forward Collision Warning (FCW) one of the foundational modules of modern Advanced Driver Assistance Systems (ADAS). This report documents an 8-week internship project carried out at KPIT Apex Lab, in which an FCW system was designed and implemented in MATLAB/Simulink using an industry-representative Model-Based Design (MBD) workflow.

The algorithm is derived from the IEEE publication *“Forward Collision Warning System Considering Both Time-to-Collision and Safety Braking Distance”*, adapted and simplified to match the scope of the internship. The Simulink model comprises three subsystems: a Safe Distance subsystem that computes the minimum and maximum safe braking distance ($D_{s,min}$, $D_{s,max}$), a Converter subsystem that derives the Time-to-Collision (TTC) and relative closing speed from the measured relative distance, and a Logic subsystem that combines these quantities against a critical TTC threshold to output a graded warning level.

In the absence of real automotive sensor logs, 300 seconds of representative simulation data were prepared in Microsoft Excel and imported into Simulink through the, enabling the model to be exercised on a realistic time-series input. The model was progressively verified through Model-in-the-Loop (MIL), Software in-the-Loop (SIL) and Processor-in-the-Loop (PIL) testing, with the auto-generated C code deployed and executed on an STM32 Nucleo(F46) development board during the PIL stage. Outputs from all three verification stages were consistent with the Simulink reference model.

1. Introduction

1.1 Context

Forward collision remains one of the most frequently reported categories of road accidents, and statistics cited in the reference literature indicate that close to 21% of collision accidents are forward collision events. Passenger vehicles are increasingly equipped with active safety systems, of which Forward Collision Warning is among the earliest and most widely deployed. An FCW system continuously monitors the distance to the vehicle ahead, estimates whether the driver retains sufficient braking margin, and issues a graded alert when that margin is reduced.

This internship project implements a simplified FCW system following the Model-Based Design methodology used across the automotive embedded software industry, in

which the control algorithm is first modelled and verified in a simulation environment before being converted to production code.

1.2 Purpose of the Study

Embedded control software in the automotive domain is rarely authored directly in C. The algorithm is first designed and simulated in MATLAB/Simulink, verified against defined requirements, and only then converted to production code through automatic code generation. This reduces the probability of defects reaching the target hardware and allows design issues to be identified at the lowest-cost stage of development. The purpose of this internship was to apply this exact methodology, end-to-end, to an FCW use case.

1.3 Objectives

- To study the working principle of a Forward Collision Warning system with reference to IEEE literature.
- To acquire working proficiency in MATLAB/Simulink and the Model-Based Design toolchain.
- To design a Simulink model computing safe braking distance, Time-to-Collision, and a graded warning output.
- To prepare a representative 300-second test dataset in Excel and interface it with Simulink.
- To verify the model through Model-in-the-Loop (MIL) simulation.
- To generate C code from the model and verify it through Software-in-the-Loop (SIL) testing.
- To deploy the generated code on an STM32 Nucleo board and verify it through Processor-in-the-Loop (PIL) testing.

1.4 Report Structure

Section 2 summarises the foundational MATLAB/Simulink training completed prior to the design phase. Section 3 presents the literature survey. Section 4 describes the system design and subsystem structure. Section 5 covers implementation details. Section 6 describes the Model-Based Design workflow followed. Sections 7, 8 and 9 present the MIL, SIL and PIL verification respectively. Section 10 discusses the results, and Section 11 concludes the report with directions for future work.

2. Training and Learning

2.1 Overview

The first week of the internship was allocated to structured training in MATLAB and Simulink prior to any design activity, using MathWorks' self-paced online courses.

2.2 MATLAB Onramp

Covered core MATLAB syntax, arrays, matrix operations, plotting, and scripting, which was subsequently used for post-processing simulation results and preparing the Excel-based test dataset.

2.3 Simulink Onramp

Introduced the Simulink block-diagram environment, signal routing, simulation execution, and result visualisation using Scope blocks, along with the concept of subsystems used to structure the FCW model.

2.4 Simulink Fundamentals

Extended the understanding of signal types, sample time configuration, and solver settings. Correct configuration of sample time was later found to be critical while interfacing the Excel-based dataset with the model.

2.5 Stateflow Onramp

Since the warning output required discrete, mutually exclusive states, Stateflow was used to implement the Logic subsystem. This course covered state definitions, transition conditions, and the interaction between Stateflow charts and Simulink signals.

2.6 Model-Based Design Basics

Established the overall MBD workflow: requirement definition, modelling, simulation, verification, code generation, and deployment. This structure was adopted directly for the remainder of the internship.

2.7 Control Design Onramp with Simulink

During the Control Design Onramp with Simulink course, the fundamentals of feedback control systems and controller design were learned using MATLAB and Simulink. Various control models were developed and simulated to understand system behaviour and validate controller performance.

2.8 Outcome

On completion of this training phase, the team was equipped to independently construct and verify the FCW model, and had a clear implementation roadmap: model, simulate, verify, generate code, deploy.

3. Literature Survey

3.1 Reference Publication

The primary reference for this project is Y.-L. Chen, K.-Y. Shen and S.-C. Wang, *“Forward Collision Warning System Considering Both Time-to-Collision and Safety Braking Distance,”* presented at the 2013 IEEE 8th Conference on Industrial Electronics and Applications (ICIEA). This section summarises the algorithm proposed in the paper and states the equations that were carried forward into the internship implementation.

3.2 Related Work Cited in the Reference Paper

The present work builds upon established research in Advanced Driver Assistance Systems (ADAS), particularly Forward Collision Warning (FCW) systems. Previous studies have demonstrated that FCW performance depends on accurate estimation of Time-to-Collision (TTC), warning distance, and vehicle braking dynamics. Chen *et al.* proposed a road-friction-based FCW algorithm that improves conventional warning strategies by incorporating variable braking deceleration under different road conditions, resulting in more realistic warning distance estimation. Additionally, recent ADAS literature highlights

FCW as a core active safety function that integrates sensor information, vehicle dynamics, and driver warning logic to reduce rear-end collisions. Motivated by these studies, the present internship project adopts a kinematic FCW approach based on TTC and safe braking distance calculations, implemented using MATLAB/Simulink and validated through Model-Based Design workflows including MIL, SIL, and PIL.

3.3 Safety Braking Distance Formulation

The reference paper expresses the braking distance $D_b(x)$ of a decelerating vehicle as

$$D_b(x) = \frac{\gamma W}{2gC_{ae}} \ln\left(1 + \frac{C_{ae} V^2}{\eta_b (\mu + f_r) W \cos \theta_s \pm W \sin \theta_s}\right)$$

where γ is the equivalent mass factor, W is vehicle weight, g is gravitational acceleration, $C_{ae} = (\rho A_f C_d)/2$ combines air density ρ , frontal area A_f and the aerodynamic drag coefficient C_d , V is vehicle speed, η_b is brake efficiency, μ is the road adhesion coefficient, f_r is the rolling resistance coefficient, and θ_s is the road slope angle, with the sign convention positive for an uphill grade and negative for a downhill grade [1].

3.4 Reaction Distance

The driver reaction time t_r is composed of reflection time, judgement time and action time, and is reported in the reference paper to lie in the range 0.74 s to 1.17 s depending on driver age. The corresponding minimum and maximum reaction distances are

$$D_{r,min} = t_{r,min} V, \quad D_{r,max} = t_{r,max} V \quad (3)$$

3.5 Pressure Build-up Distance

The distance travelled while the braking system builds up full braking pressure is

$$D_p = t_p V_0 - \frac{a_p t_p^2}{2} \quad (4)$$

which, neglecting deceleration during this short interval, reduces to

$$D_{p,min} = t_{p,min} V_0, \quad D_{p,max} = t_{p,max} V_0 \quad (5)$$

with $t_{p,min} = 0.3$ s and $t_{p,max} = 0.75$ s as reported in the reference paper.

3.6 Total Safe Braking Distance

The total safe braking distance is the sum of the three components above:

$$D_s = D_b + D_r + D_p \quad (6)$$

and, accounting for the variation in reaction time and pressure build-up time, the maximum and minimum safe braking distances are given as

$$D_{s,max} = D_b(\mu_{min}, \eta_{b,min}) + D_{r,max} + D_{p,max} \quad (7)$$

$$D_{s,min} = D_b(\mu_{max}, \eta_{b,max}) + D_{r,min} + D_{p,min} \quad (8)$$

3.7 Time-to-Collision

Time-to-Collision is defined as the relative distance divided by the rate of closure of that distance:

$$TTC = \frac{D(t)}{dD(t)/dt} \quad (9)$$

The reference paper uses a fixed critical threshold $T_c = 2.5$ s, below which the situation is classified as dangerous [1].

3.8 Warning Decision Logic

The reference paper's algorithm first compares the measured relative distance D against $D_{s,min}$ and $D_{s,max}$ to select between two decision rules, and then evaluates TTC against T_c together with the sign of dD/dt to assign a warning level, producing five levels in total (Level 0 to Level 5 except warning level 4) [1].

3.9 Adaptation for This Internship

The internship implementation retains the core computation of $D_{s,min}$, $D_{s,max}$, TTC and dD/dt , and the two-rule decision structure of Equations (9)–(8), with the following adaptations made in consultation with the mentor to fit the 8-week timeline:

- The braking distance model (Equation (1)) was reduced to depend on host vehicle speed V_{host} and road slope θ_s as primary inputs, with μ , η_b , f_r , γ and C_{ae} treated as fixed calibration constants rather than measured inputs, consistent with the parameter ranges reported in the reference paper [1].
- The warning output was reduced from five levels to four levels, retaining the same D -versus- $D_{s,min}/D_{s,max}$ and TTC -versus- T_c decision structure.
- Reaction time and pressure build-up time were retained as fixed minimum/maximum constants rather than being parameterised by driver age.
- Verification was carried out on an Excel-generated 300-second dataset rather than the vehicle-instrumented dataset used in the reference paper.

4. System Design

4.1 System Architecture

The FCW model is partitioned into three subsystems corresponding to the three computational stages of the algorithm described in Section 3: safe distance computation, distanceto-kinematics conversion, and warning decision logic.

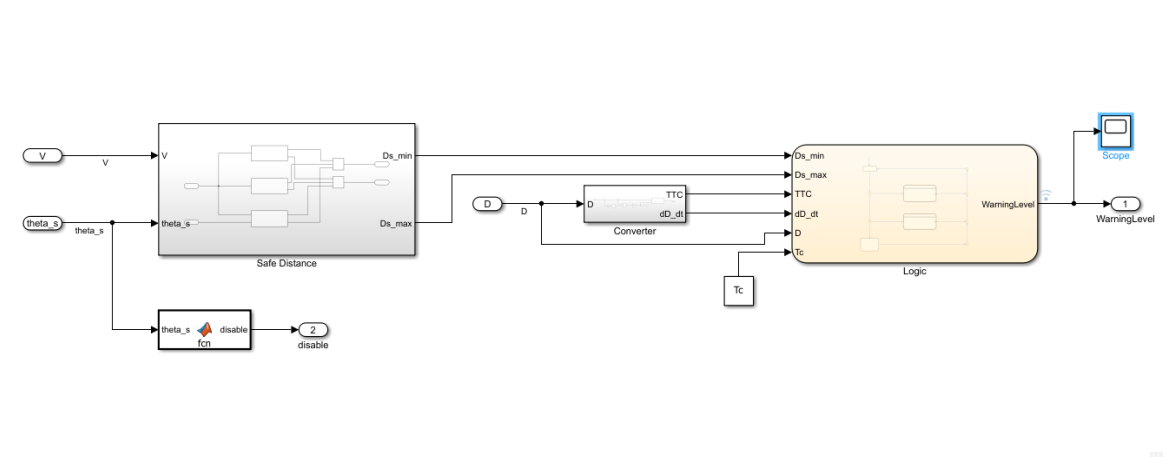


Figure 1: Top-level architecture of the Forward Collision Warning model

4.2 Subsystem 1: Safe Distance

Inputs: V_{host} (host vehicle speed), θ_s (road slope angle).

Outputs: $D_{s,min}$, $D_{s,max}$ (minimum and maximum safe braking distance).

This subsystem implements Equations (1), (3), (5) and (7)–(8), with μ , η_b , f_r , γ and C_{ae} configured as calibration constants within the ranges reported in the reference paper.

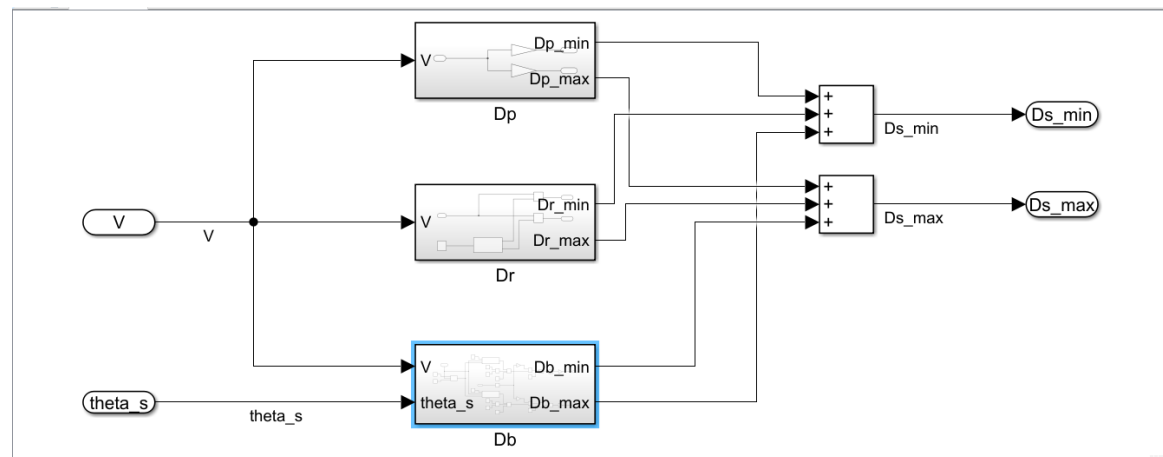


Figure 2: Safe Distance subsystem, showing computation of the minimum and maximum safe braking distance ($D_{s,min}$, $D_{s,max}$) from host vehicle speed and road slope

4.3 Subsystem 2: Converter

Input: D (relative distance to the vehicle ahead).

Outputs: TTC , dD/dt .

The relative closing speed dD/dt is obtained using a discrete derivative block operating on the incoming distance signal, and TTC is computed as per Equation (9), with a saturation limit applied to the denominator to prevent division by a near-zero closing speed.

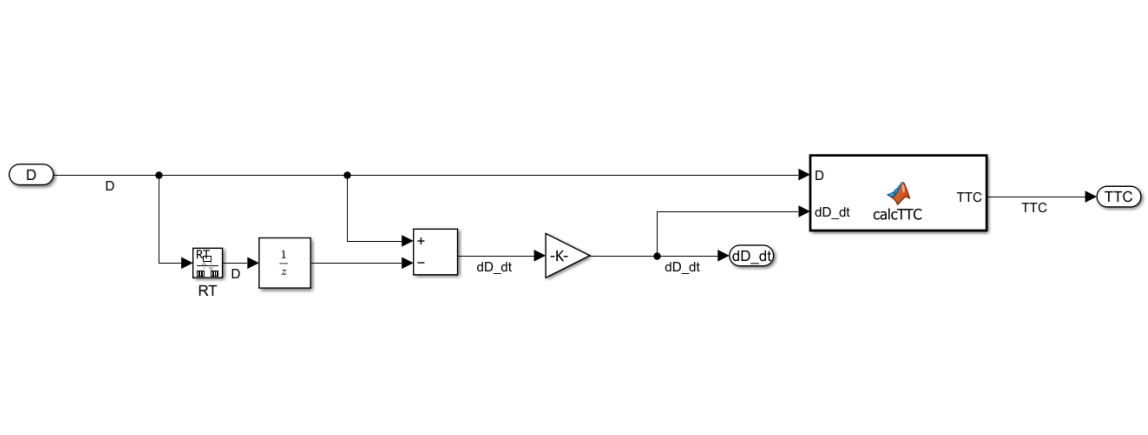


Figure 3: Converter subsystem, showing computation of Time-to-Collision (TTC) and relative closing speed (dD/dt) from the measured relative distance

4.4 Subsystem 3: Logic

Inputs: $D_{s,min}$, $D_{s,max}$, TTC , dD/dt , D , T_c .

Output: Warning Level.

Implemented as a Stateflow chart with four states, following the two-rule structure of the reference paper: D is first compared against $D_{s,min}/D_{s,max}$, and TTC is then compared against T_c together with the sign of dD/dt to select the output warning level.

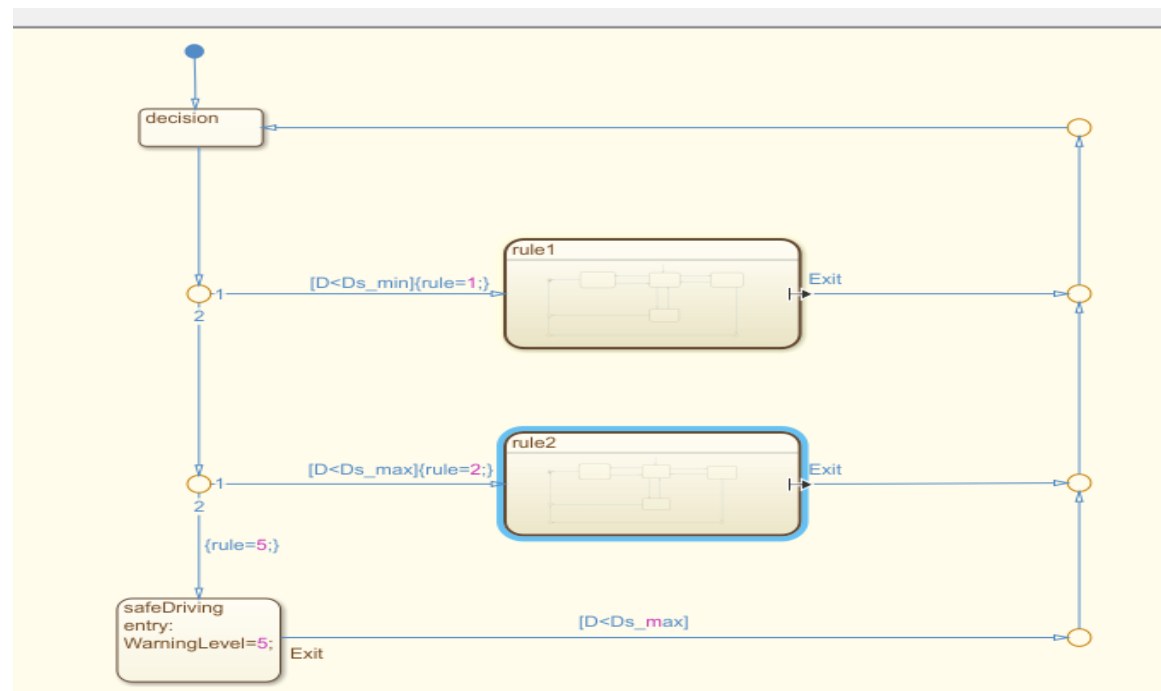


Figure 4: Logic subsystem – Stateflow warning chart

4.5 Warning Levels(Table1)

Rule 1: $D < D_{s,min}$		Rule 2: $D_{s,min} < D < D_{s,max}$		Safe Condition	
Condition	Warning Level	Condition	Warning Level	Condition	Warning Level
$TTC < T_c$	Level 3	$TTC < T_c$	Level 2	$D \geq D_{s,max}$	Level 5
$TTC \geq T_c$ and $dD/dt > 0$	Level 2	$TTC \geq T_c$ and $dD/dt > 0$	Level 1		
$TTC \geq T_c$ and $dD/dt \leq 0$	Level 1	$TTC \geq T_c$ and $dD/dt \leq 0$	Level 1		

5. Implementation

5.1 Modelling Environment

The model was implemented in MATLAB/Simulink with the Logic subsystem's decision logic embedded as a Stateflow chart, executed under a fixed-step discrete solver to be consistent with the later embedded code generation and deployment stages.

5.2 Test Dataset Preparation in Excel

In the absence of real automotive sensor logs, a 300-second representative test dataset was prepared in Microsoft Excel, consisting of a time vector along with corresponding host vehicle speed and relative distance-to-vehicle-ahead profiles. The dataset was constructed to include an acceleration phase, a steady cruising phase, a deliberate close-following interval (relative distance falling below $D_{s,min}$), and a deceleration phase, so that all four warning levels defined in Table 1 could be exercised during verification.

5.3 Excel-to-Simulink Interface

The Excel dataset was imported into the Simulink model using the model's external input configuration, where the input signals were mapped directly to the model Inport blocks through the Model Settings. This approach supplied the predefined time-varying input signals from the Excel dataset to the model throughout the simulation, enabling the same 300-second dataset to be used consistently during the MIL, SIL, and PIL validation stages.

	A	B	C	D	E	F	G	H	I	J
1	t	V	D	theta_s	Ds_min	Ds_max	Warning	dD_dt	TTC	Rule
2	0	0	15	0	0	0	5	15	1	5
3	1	0.641	16.538	0	0.691691	1.219338	5	1.538	10.752926	5
4	2	1.282	18.077	0	1.433454	2.505594	5	1.539	11.745939	5
5	3	1.923	19.615	0	2.225194	3.858602	5	1.538	12.753576	5
6	4	2.564	21.154	0	3.066755	5.278083	5	1.539	13.745289	5
7	5	3.205	22.692	0	3.95792	6.763648	5	1.538	14.754226	5
8	6	3.846	24.231	0	4.89841	8.314802	5	1.539	15.744639	5
9	7	4.487	25.769	0	5.887887	9.930947	5	1.538	16.754876	5
10	8	5.128	27.308	0	6.925954	11.61138	5	1.539	17.74399	5
11	9	5.769	28.846	0	8.012161	13.3553	5	1.538	18.755527	5
12	10	6.41	30.385	0	9.145998	15.16183	5	1.539	19.74334	5
13	11	7.051	31.923	0	10.32691	17.02997	5	1.538	20.756177	5
14	12	7.692	33.462	0	11.55427	18.95867	5	1.539	21.74269	5
15	13	8.333	35	0	12.82744	20.94678	5	1.538	22.756827	5
16	14	8.974	36.538	0	14.14571	22.99309	5	1.538	23.756827	5
17	15	9.615	38.077	0	15.50832	25.09631	5	1.539	24.741391	5
18	16	10.256	39.615	0	16.9145	27.25511	5	1.538	25.757477	5
19	17	10.897	41.154	0	18.36341	29.46808	5	1.539	26.740741	5

Figure 5: Host vehicle speed profile (Excel source, imported via From Spreadsheet block)

5.4 Safe Distance Subsystem – Implementation Detail

Constructed using Gain, Product, Trigonometric Function and Sum blocks to implement Equation (1), combined with constant blocks for the fixed reaction-time and pressure

buildup-time ranges of Equations (3) and (5). The minimum and maximum outputs are obtained by evaluating the same block structure twice, once with minimum-side and once with maximum-side calibration constants.

5.5 Converter Subsystem – Implementation Detail

Constructed using a Discrete Derivative block to obtain dD/dt and a protected Divide block to compute TTC per Equation (9).

5.6 Logic Subsystem – Implementation Detail

Constructed as a four-state Stateflow chart, with transitions governed by the comparisons in Table 1. The use of Stateflow in place of nested Switch blocks improved the readability of the decision logic and resulted in more compact generated C code.

5.7 Model Configuration

The model uses a fixed-step discrete solver, with signal data types held consistent (single precision) across all subsystem boundaries to avoid unnecessary type-conversion overhead in the generated code.

6. Model-Based Design Workflow

6.1 Requirement Analysis

Functional requirements were derived from the algorithm described in Section 3 – for example: *“the system shall output Level 3 whenever $TTC < T_c$, irrespective of relative distance.”* These requirements formed the basis for the MIL test cases in Section 7.

6.2 Modelling

Requirements were translated into the three-subsystem model described in Sections 4 and 5. Modular subsystem partitioning enabled independent verification of each stage prior to integration.

6.3 Simulation

The integrated model was simulated using the Excel-sourced 300-second dataset described in Section 5.2, with Scope blocks used to log $D_{s,min}$, $D_{s,max}$, TTC , dD/dt and the resulting warning level.

6.4 Verification

Verification followed a three-stage progression:

- MIL: the Simulink model is verified directly against requirements, without code generation.
- SIL: C code generated from the model is executed on the host machine and compared against the Simulink reference.
- PIL: the same generated code is cross-compiled and executed on the target processor (STM32 Nucleo(F446RE) board), with the plant/input model retained in Simulink.

Full detail of each stage is provided in Sections 7, 8 and 9.

6.5 Code Generation

C code was auto-generated from the Safe Distance, Converter and Logic subsystems. Since the Stateflow chart was authored with a bounded number of states and no dynamic memory allocation, the generated code required no manual modification prior to SIL and PIL testing.

lthe ek image lavu

6.6 Deployment

Deployment corresponds to the PIL stage, in which the generated controller code was executed on an STM32 Nucleo board. Integration into a production vehicle ECU was outside the scope of this internship; the PIL stage instead demonstrates functional correctness of the generated code on representative embedded hardware.

VEHICLE WARNING SYSTEM FLOWCHART

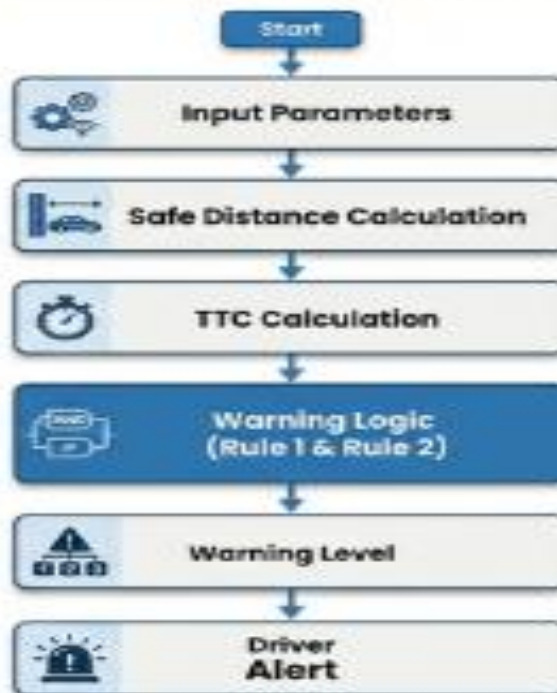


Figure 8: Model-Based Design workflow followed in this project

7. MIL Validation

7.1 Definition

Model-in-the-Loop (MIL) testing verifies the Simulink model directly, without any code generation, confirming the correctness of the algorithm prior to conversion into embedded code.

7.2 Purpose

MIL testing is performed at the earliest stage of verification because algorithmic defects identified here are the least costly to correct, and the MIL output serves as the reference against which subsequent SIL and PIL outputs are compared.

7.3 Procedure

The Excel-sourced 300-second dataset described in Section 5.2 was applied to the complete model. Output warning levels were checked against the requirements of Section

6.1 at defined intervals of the dataset, including the deliberate close-following interval where $D <$

$D_{s,min}$. All logged signals ($D_{s,min}$, $D_{s,max}$, TTC , warning level) were recorded using Scope blocks.

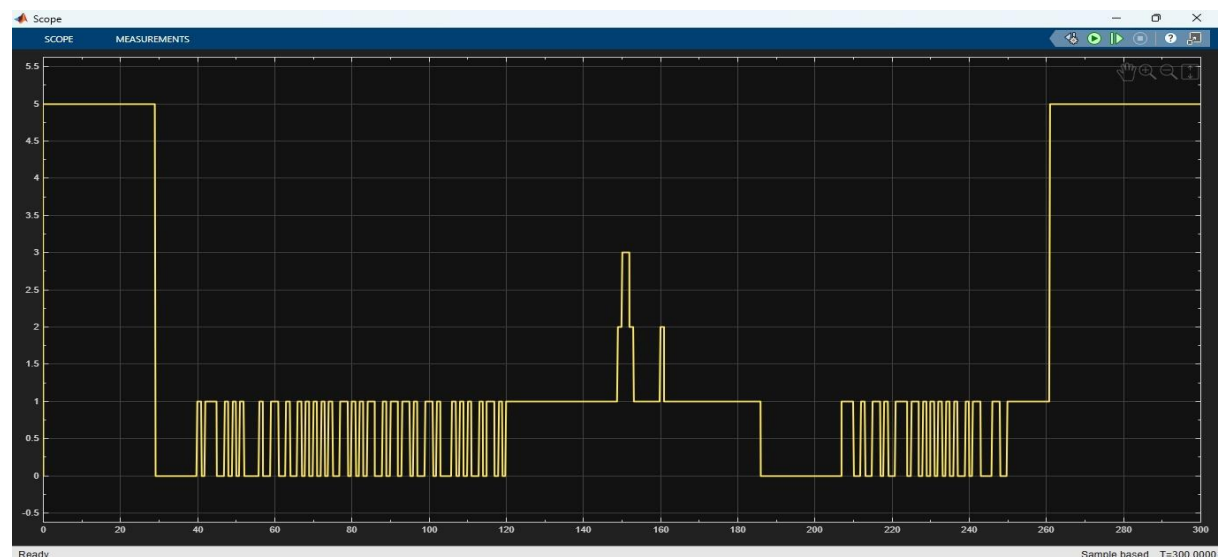


Figure 9: MIL simulation output signals

7.4 Results

The model produced the expected output across all defined scenarios: Level 0/1 during normal cruising, escalation to Level 2 as the following distance reduced, and escalation to Level 3 whenever TTC fell below T_c . MIL verification of the model was completed successfully and used as the reference dataset for SIL comparison.

8. SIL Validation

8.1 Definition

Software-in-the-Loop (SIL) testing executes the auto-generated C code as a standalone program on the host machine and compares its output against the original Simulink model, verifying that code generation has not altered functional behaviour.

8.2 Code Generation for SIL

C code was generated from the Safe Distance, Converter and Logic subsystems, with the Stateflow-based Logic subsystem mapping to a compact C function closely mirroring its state-transition structure.

8.3 Procedure

SIL was performed using the Simulink SIL/PIL Manager. The same Excel-sourced 300second dataset used for MIL testing (Section 7.3) was applied under the SIL configuration, and the SIL Manager automatically logged and compared the generated code’s output against the Simulink model output.

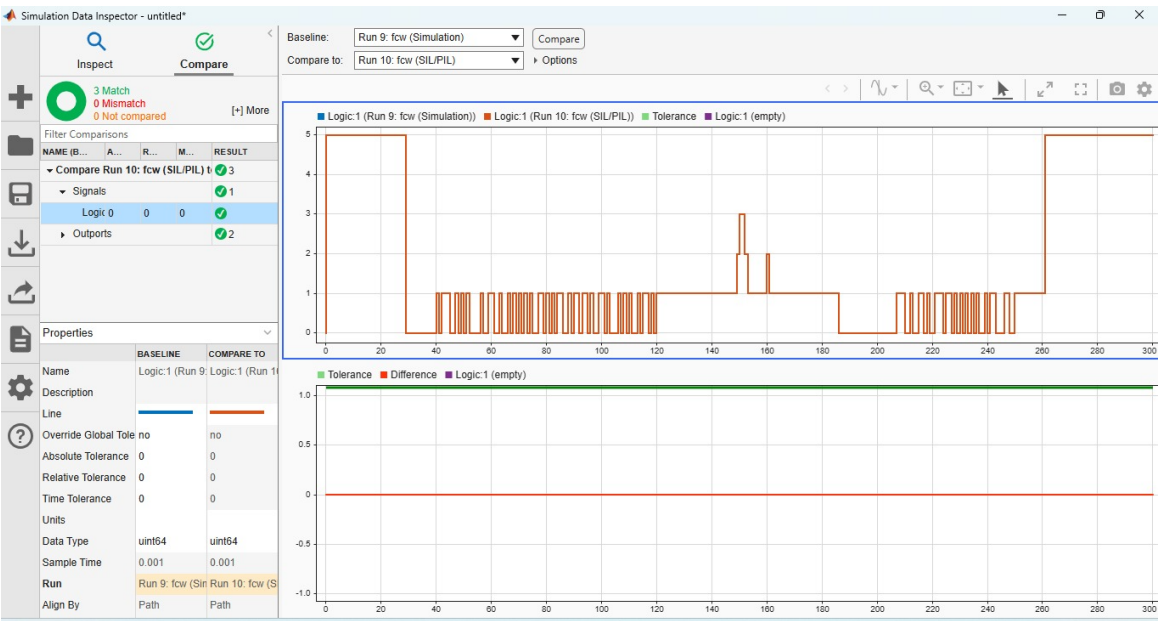


Figure 10: SIL/PIL Manager configuration

SIL comparison plot: generated code output overlaid on Simulink model output

Figure 11: SIL vs Simulink output comparison

8.4 Results

The generated C code output was compared against the Simulink model output for $D_{s,min}$, $D_{s,max}$, TTC and warning level across the full 300-second run. Outputs matched successfully, confirming that code generation did not introduce behavioural deviation from the model. SIL verification was completed successfully.

9. PIL Validation

9.1 Definition

Processor-in-the-Loop (PIL) testing executes the generated C code on the target embedded processor, while the plant/input model continues to run in Simulink on the host machine, verifying functional correctness under the target processor's actual instruction set, memory layout and numerical precision.

9.2 Target Hardware

PIL verification was carried out on an STM32 Nucleo development board. The generated controller code (Safe Distance, Converter and Logic subsystems) was deployed to the board.

9.3 Procedure

PIL was performed using the Simulink SIL/PIL Manager, configured to target the STM32 Nucleo board. The generated controller code executed on the STM32 Nucleo board while the plant model, driven by the Excel-sourced 300-second dataset, remained in Simulink on the host machine, communicating with the board over the configured communication interface.

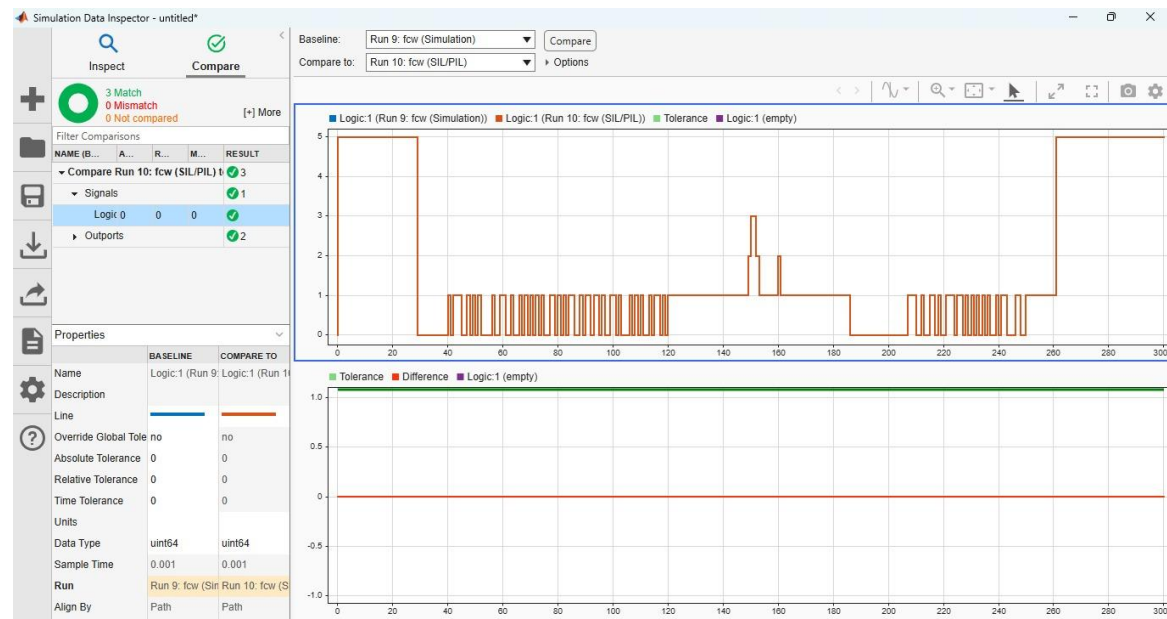


Figure 12: STM32 Nucleo board used for PIL validation

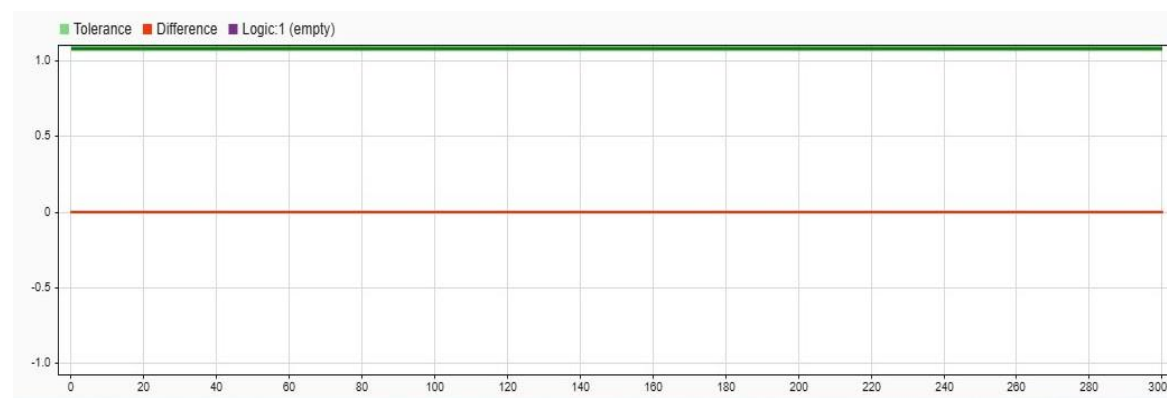


Figure 13: PIL vs Simulink output comparison

9.4 Results

Outputs generated by the code executing on the STM32 Nucleo board matched the Simulink reference outputs, confirming correct functional behaviour of the algorithm on target embedded hardware. PIL verification was completed successfully, concluding the MIL/SIL/PIL verification cycle for this project.

10. Results and Discussion

10.1 Summary of Outcomes

- Safe braking distance ($D_{s,min}$, $D_{s,max}$) computed successfully across the full range of the test speed profile.
- Time-to-Collision computed successfully from the relative distance signal.
- Warning level generated correctly for all four levels defined in Table 1.
- MIL verification completed successfully.
- SIL verification completed successfully, with generated code output matching the Simulink model.
- PIL verification completed successfully, with STM32 Nucleo output matching the Simulink model.
- Automatic C code generation completed successfully.

10.2 Consolidated Verification Summary

Table 2: Summary of verification results

Verification Stage	Status	Remarks
MIL	Successful	Output consistent with defined requirements
SIL	Successful	Generated code output matches Simulink model
PIL	Successful	STM32 Nucleo output matches Simulink model
Code Generation	Successful	No manual rework required post-generation

10.3 Discussion

The MIL-to-SIL-to-PIL progression provided verification not only that the algorithm was logically correct in simulation, but that the generated code produced identical behaviour on the host machine and on target embedded hardware. The SIL stage in particular surfaced minor data-type inconsistencies across subsystem boundaries prior to PIL deployment, which reduced the debugging effort required at the hardware stage.

Relative to the reference publication [1], the reduced-parameter braking distance model and four-level warning scheme were sufficient to reproduce the underlying safety principle – comparing relative distance against safe braking distance, and TTC against a critical threshold – while remaining within the scope achievable in an 8-week internship.

10.4 Limitations

- **Excel-Based Inputs:** The proposed FCW system utilizes predefined Excel datasets for host vehicle velocity, relative distance, and road slope instead of acquiring real-time sensor measurements
- **Single Vehicle Tracking:** The implemented model monitors only a single lead vehicle traveling directly ahead of the host vehicle and does not consider multiple surrounding vehicles
- **Simplified Vehicle Dynamics:** The vehicle model considers only longitudinal motion and does not account for lateral dynamics, lane changes, or steering maneuvers.
- **No Automatic Braking:** The developed system is limited to generating collision warning levels and does not implement Automatic Emergency Braking (AEB) or other vehicle control actions.
- **Ideal Sensor Assumption:** Sensor measurements are assumed to be accurate, reliable, and pre-filtered, while sensor noise, communication delays, and measurement uncertainties are neglected.

11. Conclusion and Future Scope

11.1 Conclusion

This internship at KPIT Apex Lab provided practical exposure to the complete ModelBased Design workflow used in automotive embedded software development, applied to a Forward Collision Warning system based on the algorithm proposed in [1]. Progressing from MATLAB/Simulink fundamentals through requirement analysis, subsystem design, Excel-based dataset integration, and MIL/SIL/PIL verification, the generated code was successfully deployed and executed on an STM32 Nucleo board, with results consistent across all verification stages.

11.2 Future Scope

- **Adaptive Cruise Control (ACC):** Integrate the FCW system with Adaptive Cruise Control to automatically maintain a safe following distance from the lead vehicle..

- **Automatic Emergency Braking (AEB):** Extend the current warning system by incorporating an Automatic Emergency Braking controller to autonomously apply brakes in critical collision scenarios..
- **Radar Sensor Fusion:** Integrate radar sensor data to obtain accurate real-time measurements of relative distance and relative velocity, improving the reliability of collision detection..
- **Camera Sensor Fusion:** Combine camera-based object detection with radar data to enhance obstacle recognition, lane awareness, and overall environmental perception..
- **Extended Hardware-in-the-Loop (HIL) Validation:** Validate the complete FCW/AEB system using a real-time Hardware-in-the-Loop platform to assess controller performance under realistic driving scenarios before deployment

Bibliography

- [1] Y.-L. Chen, K.-Y. Shen, and S.-C. Wang, "Forward Collision Warning System Considering Both Time-to-Collision and Safety Braking Distance," in *2013 IEEE 8th Conference on Industrial Electronics and Applications (ICIEA)*, pp. 972–977, 2013.
- [2] T. Chen, K. Liu, Z. Wang, G. Deng, and B. Chen, "Vehicle forward collision warning algorithm based on road friction," *Transportation Research Part D: Transport and Environment*, vol. 66, pp. 49–57, Jan. 2019.
- [3] *Advanced Driver Assistance Systems (ADAS)*, Technical Review Report, 2024.
- [4] S. K. Gehrig and F. J. Stein, "Collision Avoidance for Vehicle-Following Systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 8, no. 2, pp. 233–244, June 2007.
- [5] A. Vahidi and A. Eskandarian, "Research Advances in Intelligent Collision Avoidance and Adaptive Cruise Control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 4, no. 3, pp. 143–153, September 2003.
- [6] R. V. Dukkipati, J. Pang, M. S. Qatu, and Z. Shuguang, *Road Vehicle Dynamics*, SAE International, 2008.
- [7] MathWorks, *MATLAB Onramp and Simulink Onramp*, self-paced online courses. Available: <https://matlabacademy.mathworks.com>
- [8] MathWorks, *Model-Based Design Basics*, self-paced online course. Available: <https://matlabacademy.mathworks.com>
- [9] MathWorks, *SIL and PIL Simulations*, Simulink documentation. Available: <https://www.mathworks.com/help/ecoder/sil-and-pil-simulations.html>

-
- [10] STMicroelectronics, *STM32 Nucleo Development Boards*, product documentation. Available: <https://www.st.com/en/evaluation-tools/nucleo-boards.html>

A.1. Notation

Table 3: List of symbols used in this report

Symbol	Description
V_{host}, V	Host vehicle speed
θ_s	Road slope angle
D	Relative distance between host vehicle and vehicle ahead
$D_{s,min}, D_{s,max}$	Minimum / maximum safe braking distance
D_b	Braking distance
D_r	Reaction distance
D_p	Pressure build-up distance
γ	Equivalent mass factor
W	Vehicle weight
g	Gravitational acceleration
C_{ae}	Aerodynamic drag term ($\rho A_f C_d / 2$)
η_b	Brake efficiency
μ	Road adhesion coefficient
f_r	Rolling resistance coefficient
s_r	Wheel slip ratio
t_r, t_p	Reaction time, pressure build-up time
TTC	Time-to-Collision
T_c	Critical TTC threshold
dD/dt	Relative closing speed

A.2. Excel Dataset Structure

Table 4: Structure of the 300-second Excel test dataset

Column	Description
Time (s)	Simulation time vector, 0 to 300 s
Host Speed (km/h)	Host vehicle speed profile
Relative Distance (m)	Distance to vehicle ahead
Road Slope (deg)	Road slope angle

A.3. Generated C Code (Excerpt)

```

48
49  /* External outputs (root outputs fed by signals with default storage) */
50  extv rtv;
51
52  /* Real-time model */
53  static RT_MODEL rtM;
54  RT_MODEL *const rtM = &rtM;
55  extern real_T rt_read_snf(real_T u0, real_T u1);
56  static void MATLABFunction(real_T rtu_M, real_T rtu_theta_s, boolean_T
57    rtu_uphi11, real_T *rtu_theta_term);
58
59  /* forward declaration for local functions */
60  static void cosd(real_T *X);
61  static void sind(real_T *X);
62  static void rate_monotonic_scheduler(void);
63  static real_T rtGetNaN(void);
64  static real32_T rtGetNaNF(void);
65  extern real_T rtInff;
66  extern real_T rtMinusInff;
67  extern real_T rtNaN;
68  extern real32_T rtInff;
69  extern real32_T rtMinusInff;
70  extern real32_T rtNaNF;
71  static boolean_T rtIsInff(real_T value);
72  static boolean_T rtIsInff(real32_T value);
73  static boolean_T rtIsNaN(real_T value);

```

Diagnostic Viewer

```

10:44 AM: Simulation, S
Filter
0 0 0 9
C:\Users\swpn11\AppData\Local\Temp\6d9b93c6-12cc-4f75-9742-
b3923b11cb12_STM_ioc.rar.b12\MATLAB\fcw_ert_rtw\sil\exit /B 0
### Starting SIL simulation for component: fwc
### Application stopped
### Stopping SIL simulation for component: fwc

```

Logged signals from the two simulations are compared in Simulation Data Inspector.

Ready View diagnostics 100% FixedStepDiscrete

Figure 14: Excerpt of generated C code

